

Introduction to Segmentation

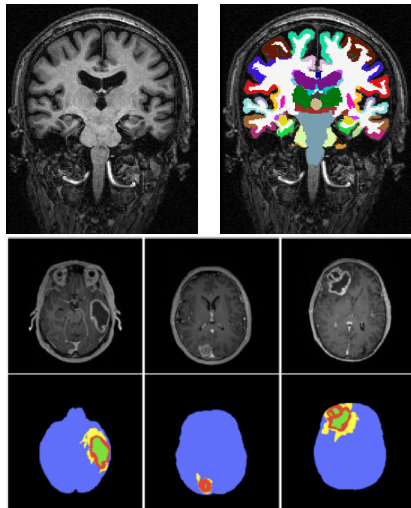
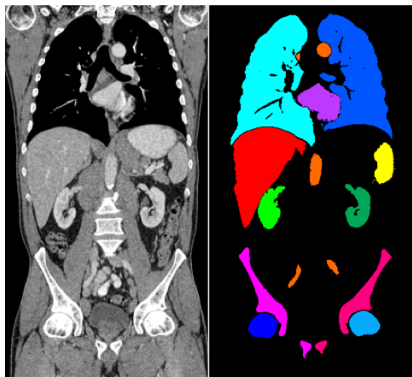
Ender Konukoglu

ETH Zürich

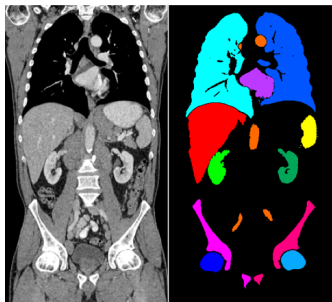
March 24, 2026

Segmentation

From image intensities to anatomical structures and semantic information



Segmentation principle



- At each pixel / voxel assign a label
- Labels can be
 - Organ: liver, spine, ...
 - Part of organs: individual vertebrae, ...
 - Lesions: tumors, pathologies, ...
- Information at each pixel
 - Intensity at and around the pixel
 - Intensity at different modalities

$x \in \Omega \subset \mathbb{R}^d$, $d = 2$ or $d = 3$

$f(x) = [I(x), I(\mathcal{N}(x)), J(\mathcal{N}(x)), \dots]$: Features

$L(x) \in \{0, \dots, N\}$: Labels

$L(x) \approx S(f(x))$: Segmentation approximating labels

Segmentation techniques overview

- Thresholding and histogram
- Clustering
 - K-means
 - Unsupervised learning
 - Non-parametric modeling
 - ...
- Graph partitioning methods
 - Watershed
 - Graph-cuts
 - Random walker
 - Minimum spanning forest
 - ...
- Region growing
- Variational and PDE-based
 - Chan-Vese model
 - Mumford-Shah model
 - Active-shape models
 - Level sets
 - Fast marching
 - ...
- Discriminative modeling based
 - Random forest
 - Conditional random fields
 - Supervised convolutional neural networks
 - ...
- Generative modeling based
 - Expectation-maximization
 - Markov random fields
 - Atlas-based segmentation
 - Variational inference
 - ...

Overview

1 Basic Segmentation

- Thresholding
- K-Means Clustering
- Expectation-Maximization Segmentation

2 Atlas-based segmentation

- Formulation
- Multi-atlas segmentation

3 Spatial constraints in segmentation

- Morphological operations
- Random field priors

Section 1

Basic Segmentation

Subsection 1

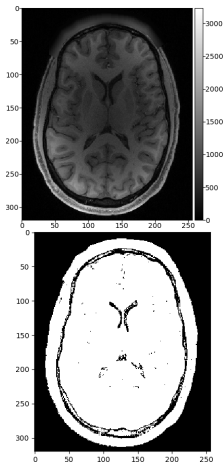
Thresholding

Thresholding

■ Simple model

$$L(x) = \begin{cases} 1, & I(x) > \tau \\ 0, & I(x) \leq \tau \end{cases}$$

or vice-versa.



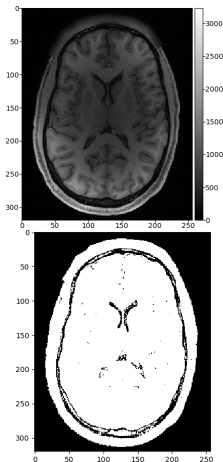
Thresholding

- Simple model

$$L(x) = \begin{cases} 1, & I(x) > \tau \\ 0, & I(x) \leq \tau \end{cases}$$

or vice-versa.

- Particularly useful in easy foreground-background subtraction



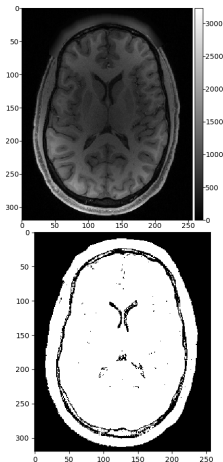
Thresholding

- Simple model

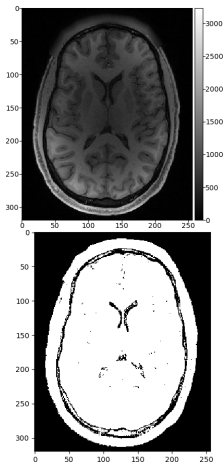
$$L(x) = \begin{cases} 1, & I(x) > \tau \\ 0, & I(x) \leq \tau \end{cases}$$

or vice-versa.

- Particularly useful in easy foreground-background subtraction
- Preprocessing method for region of interest determination



Thresholding

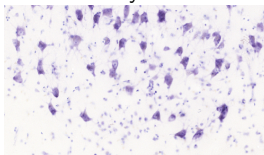


■ Simple model

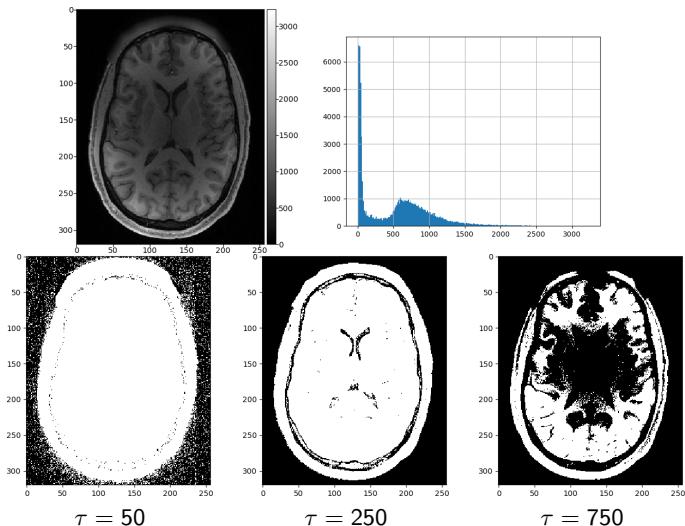
$$L(x) = \begin{cases} 1, & I(x) > \tau \\ 0, & I(x) \leq \tau \end{cases}$$

or vice-versa.

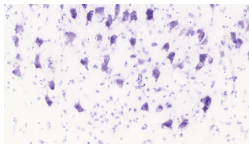
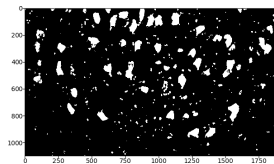
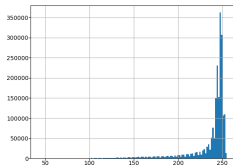
- Particularly useful in easy foreground-background subtraction
- Preprocessing method for region of interest determination
- Used routinely in histology and other microscopic images where stains provide the necessary contrast



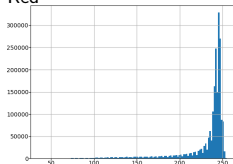
How to determine the threshold: histogram analysis



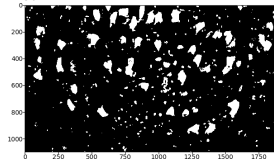
How to determine the threshold: histogram analysis



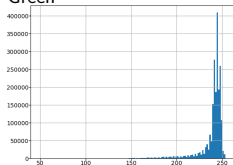
Red



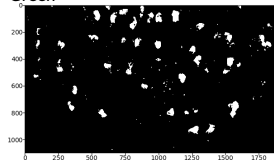
Red



Green



Green



Blue

Blue

How to determine the threshold: automatic methods

- Methods to determine peaks and troughs of the histogram
- Otsu's thresholding method

$$\min p_1\sigma_1^2 + p_2\sigma_2^2$$

where $p_{1,2}$ are the number of pixels in fore and background. $\sigma_{1,2}^2$ are the intensity variances in the groups.

How to determine the threshold: automatic methods

- Methods to determine peaks and troughs of the histogram
- Otsu's thresholding method

$$\min p_1\sigma_1^2 + p_2\sigma_2^2$$

where $p_{1,2}$ are the number of pixels in fore and background. $\sigma_{1,2}^2$ are the intensity variances in the groups.

Further remarks:

- Global thresholding (for the entire image) or local thresholding (per ROI or patch)
- Image noise can lead to isolated FG or BG islands
- Multiple objects would require multiple thresholds
- A nice generalization is K-means algorithm

Subsection 2

K-Means Clustering

K-Means segmentation

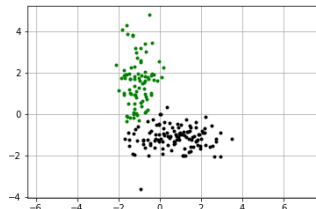
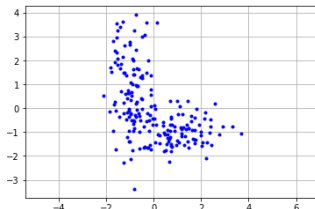
- Unsupervised clustering algorithm

K-Means segmentation

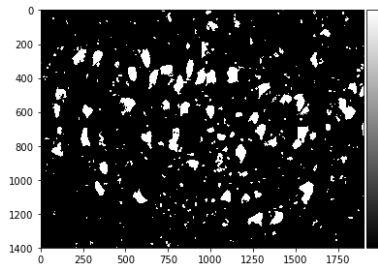
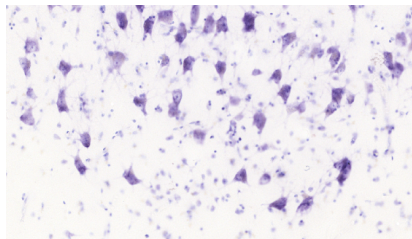
- Unsupervised clustering algorithm
- Voxels/pixels are clustered according to *features*
 - intensity
 - color
 - temporal sequence
 - other features, e.g. gradient, wavelet transform,...

K-Means segmentation

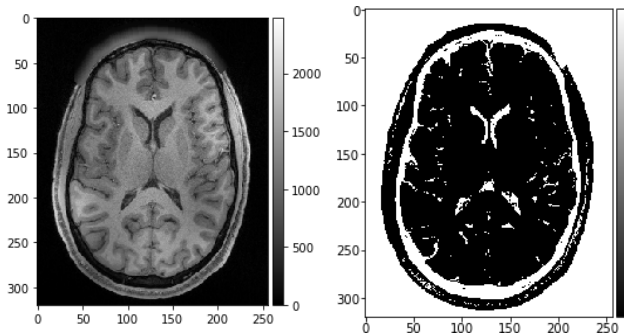
- Unsupervised clustering algorithm
 - Voxels/pixels are clustered according to *features*
 - intensity
 - color
 - temporal sequence
 - other features, e.g. gradient, wavelet transform,...
 - Automatically assigns each voxel/pixel to one of N clusters
- Assume we have two features (shown in x and y axis), multiple pixels (each point) and two clusters



K-Means segmentation - 2 clusters, features=RGB



K-Means segmentation - 2 clusters, features=intensity



How does it work?

- We want to distribute pixels to N groups using features

How does it work?

- We want to distribute pixels to N groups using features
- Criteria:
 - Homogeneity within groups
 - Reducing variance over features within group
 - Take into account multiple features

How does it work?

- We want to distribute pixels to N groups using features
- Criteria:
 - Homogeneity within groups
 - Reducing variance over features within group
 - Take into account multiple features
- Unsupervised - no information on the structures nor groups

How does it work?

- We want to distribute pixels to N groups using features
- Criteria:
 - Homogeneity within groups
 - Reducing variance over features within group
 - Take into account multiple features
- Unsupervised - no information on the structures nor groups
- Iterative algorithm that assigns pixels to groups, computes group means and reassigns based on distance to group centers

K-Means Algorithm

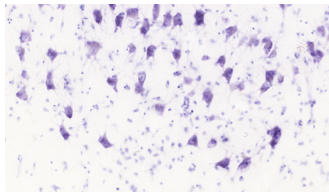
Algorithm 1 K-Means

- 1: Describe each pixel with features $f(x) \in \mathbb{R}^m$, e.g. for only intensity $m = 1$ and for RGB $m = 3$
- 2: Randomly choose N points as the group centers, $m_i^0 \in \mathbb{R}^m$, $i = 1, \dots, N$
- 3: **while** $\|m_i^t - m_i^{t-1}\| > \epsilon$ for any i **do**
- 4: Assign groups: $c(x) = \arg_i \min \|f(x) - m_i\|_2$
- 5: Recompute means:

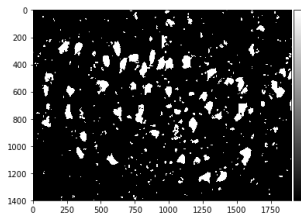
$$m_i = \frac{\sum_{x \in \Omega} \delta_{i=c(x)} f(x)}{\sum_{x \in \Omega} \delta_{i=c(x)}}, \quad \delta_{i=c(x)} = \begin{cases} 0, & i \neq c(x) \\ 1, & i = c(x) \end{cases}$$

- 6: **end while**
-

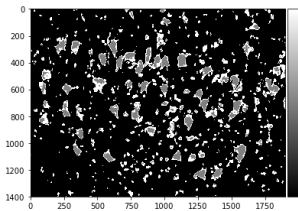
Changing number of clusters



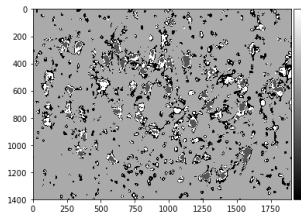
image



2 clusters

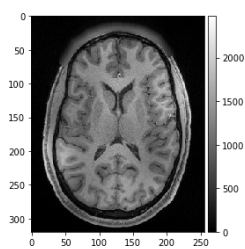


3 clusters

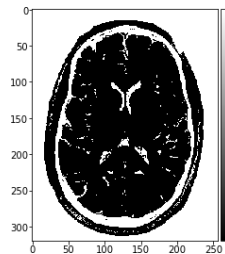


4 clusters

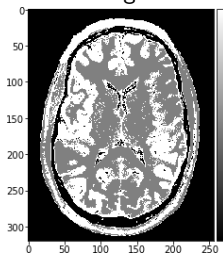
Changing number of clusters



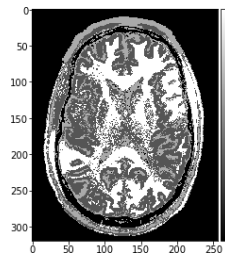
image



2 clusters

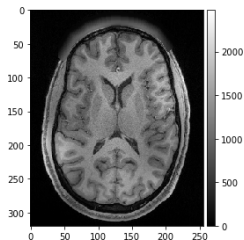


3 clusters

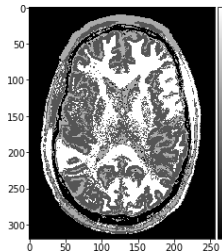


4 clusters

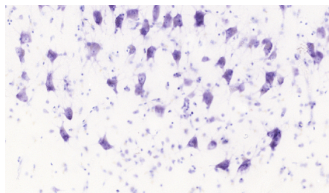
Question: Why does this happen?



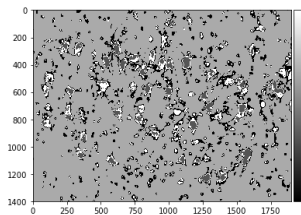
image



4 clusters



image



4 clusters

Remarks

- It aims to create clusters that have as homogeneous as possible features within clusters.

Remarks

- It aims to create clusters that have as homogeneous as possible features within clusters.

Aims to find cluster assignments and mean values that explain the data as best as possible with the given number of clusters.

Remarks

- It aims to create clusters that have as homogeneous as possible features within clusters.
Aims to find cluster assignments and mean values that explain the data as best as possible with the given number of clusters.
- Very easy to implement
- Choice of number of clusters has a major influence

Remarks

- It aims to create clusters that have as homogeneous as possible features within clusters.

Aims to find cluster assignments and mean values that explain the data as best as possible with the given number of clusters.

- Very easy to implement
- Choice of number of clusters has a major influence
- Methods exist to choose it automatically:
 - Heuristic methods based on variance
 - Non-parametric Bayesian methods

Remarks

- It aims to create clusters that have as homogeneous as possible features within clusters.

Aims to find cluster assignments and mean values that explain the data as best as possible with the given number of clusters.

- Very easy to implement
- Choice of number of clusters has a major influence
- Methods exist to choose it automatically:
 - Heuristic methods based on variance
 - Non-parametric Bayesian methods
- Initialization is important! K-Means can get stuck in local minima:
 - Run it multiple times with different initializations
 - Multi-scale methods for robustness

Remarks

- It aims to create clusters that have as homogeneous as possible features within clusters.
Aims to find cluster assignments and mean values that explain the data as best as possible with the given number of clusters.
- Very easy to implement
- Choice of number of clusters has a major influence
- Methods exist to choose it automatically:
 - Heuristic methods based on variance
 - Non-parametric Bayesian methods
- Initialization is important! K-Means can get stuck in local minima:
 - Run it multiple times with different initializations
 - Multi-scale methods for robustness
- Probabilistic formulation is possible that can be more robust and allows extensions,

Remarks

- It aims to create clusters that have as homogeneous as possible features within clusters.

Aims to find cluster assignments and mean values that explain the data as best as possible with the given number of clusters.

- Very easy to implement
- Choice of number of clusters has a major influence
- Methods exist to choose it automatically:
 - Heuristic methods based on variance
 - Non-parametric Bayesian methods
- Initialization is important! K-Means can get stuck in local minima:
 - Run it multiple times with different initializations
 - Multi-scale methods for robustness
- Probabilistic formulation is possible that can be more robust and allows extensions, e.g.,

$$\max \ln p(I).$$

Subsection 3

Expectation-Maximization Segmentation

Mixture Model

- We will work on the same idea as K-Means but take a probabilistic view.

Mixture Model

- We will work on the same idea as K-Means but take a probabilistic view.
- This will allow us to create accurate *atlas*-based segmentation methods and motivate registration.

Mixture Model

- We will work on the same idea as K-Means but take a probabilistic view.
- This will allow us to create accurate *atlas*-based segmentation methods and motivate registration.
- Start by assuming all pixels are independent and model the intensities as a **mixture model**

Mixture Model

- We will work on the same idea as K-Means but take a probabilistic view.
- This will allow us to create accurate *atlas*-based segmentation methods and motivate registration.
- Start by assuming all pixels are independent and model the intensities as a **mixture model**
- Mixture model for a given feature vector f

$$p(f) = \sum_{n=1}^N p(f|c = n)p(c = n)$$

$p(c)$ is the *prior probability* of observing (latent) label c . Also called **mixture coefficients**.

Mixture Model

- We will work on the same idea as K-Means but take a probabilistic view.
- This will allow us to create accurate *atlas*-based segmentation methods and motivate registration.
- Start by assuming all pixels are independent and model the intensities as a **mixture model**
- Mixture model for a given feature vector f

$$p(f) = \sum_{n=1}^N p(f|c = n)p(c = n)$$

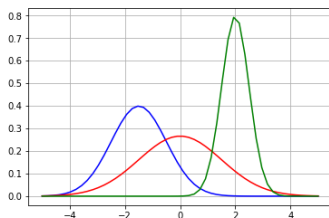
$p(c)$ is the *prior probability* of observing (latent) label c . Also called **mixture coefficients**.

- $p(f|c)$ is the *likelihood model* defined at that point. Often taken as a Gaussian

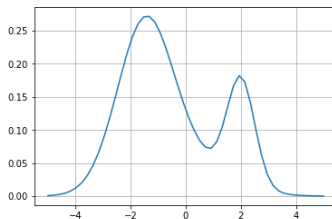
$$p(f|c = n) = \mathcal{N}(f|\mu_n, \Sigma_n) = \frac{1}{\sqrt{2\pi|\Sigma_n|}} \exp\left(-\frac{1}{2}(f - \mu_n)^T \Sigma_n^{-1}(f - \mu_n)\right)$$

μ_n and Σ_n are class specific parameters $\rightarrow$ **Gaussian mixture model**

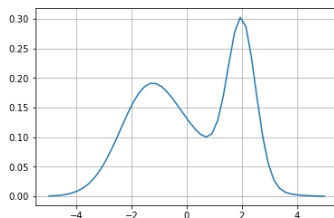
Mixture Model - 1D Example with 3 (latent) labels



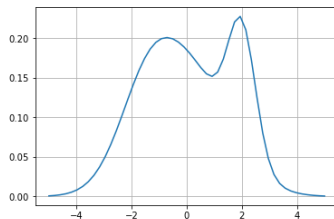
Components



$$p(c = 0) = 3/5, \quad p(c = 1) = p(c = 2) = 1/5$$

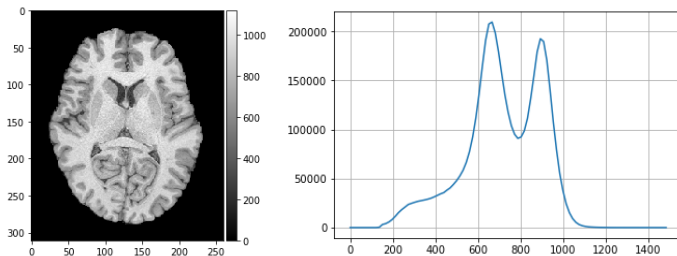


$$p(c = 0) = p(c = 1) = p(c = 2) = 1/3$$



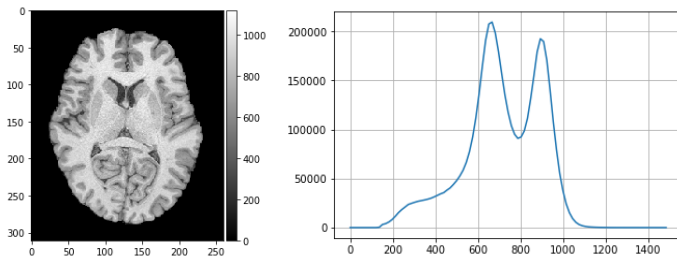
$$p(c = 1) = 3/5, \quad p(c = 0) = p(c = 2) = 1/5$$

Does it look like real image distribution?



Left is the image and right shows the intensity histogram.

Does it look like real image distribution?



Left is the image and right shows the intensity histogram.

Gaussian Mixture Model may not be a bad approximate model for the intensities

How to use the mixture model?

- Fit the model parameters to the image intensities in an unsupervised way, we will only assume a number of classes N and assume intensities at different pixels are independent from each other and they are the features to which we will fit the model

$$\max_{\theta} \log p(I|\theta) = \max_{\theta} \log \prod_{x \in \Omega} p(I(x)|\theta) = \max_{\theta} \sum_{x \in \Omega} \log p(I(x)|\theta)$$

How to use the mixture model?

- Fit the model parameters to the image intensities in an unsupervised way, we will only assume a number of classes N and assume intensities at different pixels are independent from each other and they are the features to which we will fit the model

$$\max_{\theta} \log p(I|\theta) = \max_{\theta} \log \prod_{x \in \Omega} p(I(x)|\theta) = \max_{\theta} \sum_{x \in \Omega} \log p(I(x)|\theta)$$

$$\log p(I(x)|\theta) = \log \sum_{n=1}^N p(I(x)|c(x) = n)p(c(x) = n)$$

How to use the mixture model?

- Fit the model parameters to the image intensities in an unsupervised way, we will only assume a number of classes N and assume intensities at different pixels are independent from each other and they are the features to which we will fit the model

$$\max_{\theta} \log p(I|\theta) = \max_{\theta} \log \prod_{x \in \Omega} p(I(x)|\theta) = \max_{\theta} \sum_{x \in \Omega} \log p(I(x)|\theta)$$

$$\log p(I(x)|\theta) = \log \sum_{n=1}^N p(I(x)|c(x) = n)p(c(x) = n)$$

θ is the set of model parameters:

- $p(c(x) = n) = \pi_n$: class probabilities
- μ_n, Σ_n : likelihood parameters

How to use the mixture model?

- Fit the model parameters to the image intensities in an unsupervised way, we will only assume a number of classes N and assume intensities at different pixels are independent from each other and they are the features to which we will fit the model

$$\max_{\theta} \log p(I|\theta) = \max_{\theta} \log \prod_{x \in \Omega} p(I(x)|\theta) = \max_{\theta} \sum_{x \in \Omega} \log p(I(x)|\theta)$$

$$\log p(I(x)|\theta) = \log \sum_{n=1}^N p(I(x)|c(x)=n)p(c(x)=n)$$

θ is the set of model parameters:

- $p(c(x)=n) = \pi_n$: class probabilities
- μ_n, Σ_n : likelihood parameters
- After fitting, **segmentation is defined through posterior-distribution using optimal parameters**

$$p(c(x)|I(x)) \quad c^*(x) = \arg \max p(c(x)|I(x))$$

How to use the mixture model?

- Fit the model parameters to the image intensities in an unsupervised way, we will only assume a number of classes N and assume intensities at different pixels are independent from each other and they are the features to which we will fit the model

$$\max_{\theta} \log p(I|\theta) = \max_{\theta} \log \prod_{x \in \Omega} p(I(x)|\theta) = \max_{\theta} \sum_{x \in \Omega} \log p(I(x)|\theta)$$

$$\log p(I(x)|\theta) = \log \sum_{n=1}^N p(I(x)|c(x) = n) p(c(x) = n)$$

θ is the set of model parameters:

- $p(c(x) = n) = \pi_n$: class probabilities
- μ_n, Σ_n : likelihood parameters
- After fitting, **segmentation is defined through posterior-distribution using optimal parameters**

$$p(c(x)|I(x)) \quad c^*(x) = \arg \max p(c(x)|I(x))$$

- We can optimize using *gradient descent* - possible but difficult to optimize

How to use the mixture model?

- Fit the model parameters to the image intensities in an unsupervised way, we will only assume a number of classes N and assume intensities at different pixels are independent from each other and they are the features to which we will fit the model

$$\max_{\theta} \log p(I|\theta) = \max_{\theta} \log \prod_{x \in \Omega} p(I(x)|\theta) = \max_{\theta} \sum_{x \in \Omega} \log p(I(x)|\theta)$$

$$\log p(I(x)|\theta) = \log \sum_{n=1}^N p(I(x)|c(x) = n) p(c(x) = n)$$

θ is the set of model parameters:

- $p(c(x) = n) = \pi_n$: class probabilities
- μ_n, Σ_n : likelihood parameters
- After fitting, **segmentation is defined through posterior-distribution using optimal parameters**

$$p(c(x)|I(x)) \quad c^*(x) = \arg \max p(c(x)|I(x))$$

- We can optimize using *gradient descent* - possible but difficult to optimize
- Better alternative: Expectation-Maximization

Key idea - two alternating steps

- **E-step** - Expectation-step: Assume a set of likelihood parameters and "soft" assign samples to labels
- **M-step** - Maximization-step: Assume soft assignments and maximize likelihood parameters, e.g. find the best mean and standard deviations of the Gaussians.

Expectation-Maximization - derivation

Expectation-Maximization - derivation

Dropping the dependence on x for simplicity for now and using Bayes' rule

Expectation-Maximization - derivation

Dropping the dependence on x for simplicity for now and using Bayes' rule

$$\log p(I|\theta) = \log p(I, c|\theta) - \log p(c|I, \theta)$$

Expectation-Maximization - derivation

Dropping the dependence on x for simplicity for now and using Bayes' rule

$$\log p(I|\theta) = \log p(I, c|\theta) - \log p(c|I, \theta)$$

Let us assume we are given an θ_{old} , then

$$\mathbb{E}_{p(c|I, \theta_{old})}[\log p(I|\theta)] = \mathbb{E}_{p(c|I, \theta_{old})}[\log p(I, c|\theta)] - \mathbb{E}_{p(c|I, \theta_{old})}[\log p(c|I, \theta)]$$

Expectation-Maximization - derivation

Dropping the dependence on x for simplicity for now and using Bayes' rule

$$\log p(I|\theta) = \log p(I, c|\theta) - \log p(c|I, \theta)$$

Let us assume we are given an θ_{old} , then

$$\begin{aligned}\mathbb{E}_{p(c|I, \theta_{old})}[\log p(I|\theta)] &= \mathbb{E}_{p(c|I, \theta_{old})}[\log p(I, c|\theta)] - \mathbb{E}_{p(c|I, \theta_{old})}[\log p(c|I, \theta)] \\ \log p(I|\theta) &= \mathbb{E}_{p(c|I, \theta_{old})}[\log p(I, c|\theta)] - \mathbb{E}_{p(c|I, \theta_{old})}[\log p(c|I, \theta)]\end{aligned}$$

Expectation-Maximization - derivation

Dropping the dependence on x for simplicity for now and using Bayes' rule

$$\log p(I|\theta) = \log p(I, c|\theta) - \log p(c|I, \theta)$$

Let us assume we are given an θ_{old} , then

$$\begin{aligned}\mathbb{E}_{p(c|I, \theta_{old})}[\log p(I|\theta)] &= \mathbb{E}_{p(c|I, \theta_{old})}[\log p(I, c|\theta)] - \mathbb{E}_{p(c|I, \theta_{old})}[\log p(c|I, \theta)] \\ \log p(I|\theta) &= \mathbb{E}_{p(c|I, \theta_{old})}[\log p(I, c|\theta)] - \mathbb{E}_{p(c|I, \theta_{old})}[\log p(c|I, \theta)] \\ &= Q(\theta|\theta_{old}) - \mathbb{E}_{p(c|I, \theta_{old})}[\log p(c|I, \theta)]\end{aligned}$$

Expectation-Maximization - derivation

Dropping the dependence on x for simplicity for now and using Bayes' rule

$$\log p(I|\theta) = \log p(I, c|\theta) - \log p(c|I, \theta)$$

Let us assume we are given an θ_{old} , then

$$\begin{aligned}\mathbb{E}_{p(c|I, \theta_{old})}[\log p(I|\theta)] &= \mathbb{E}_{p(c|I, \theta_{old})}[\log p(I, c|\theta)] - \mathbb{E}_{p(c|I, \theta_{old})}[\log p(c|I, \theta)] \\ \log p(I|\theta) &= \mathbb{E}_{p(c|I, \theta_{old})}[\log p(I, c|\theta)] - \mathbb{E}_{p(c|I, \theta_{old})}[\log p(c|I, \theta)] \\ &= Q(\theta|\theta_{old}) - \mathbb{E}_{p(c|I, \theta_{old})}[\log p(c|I, \theta)]\end{aligned}$$

One can show that

$$0 \geq \mathbb{E}_{p(c|I, \theta_{old})}[\log p(c|I, \theta_{old})] \geq \mathbb{E}_{p(c|I, \theta_{old})}[\log p(c|I, \theta)], \quad \forall \theta$$

Expectation-Maximization - derivation

Dropping the dependence on x for simplicity for now and using Bayes' rule

$$\log p(I|\theta) = \log p(I, c|\theta) - \log p(c|I, \theta)$$

Let us assume we are given an θ_{old} , then

$$\begin{aligned}\mathbb{E}_{p(c|I, \theta_{old})}[\log p(I|\theta)] &= \mathbb{E}_{p(c|I, \theta_{old})}[\log p(I, c|\theta)] - \mathbb{E}_{p(c|I, \theta_{old})}[\log p(c|I, \theta)] \\ \log p(I|\theta) &= \mathbb{E}_{p(c|I, \theta_{old})}[\log p(I, c|\theta)] - \mathbb{E}_{p(c|I, \theta_{old})}[\log p(c|I, \theta)] \\ &= Q(\theta|\theta_{old}) - \mathbb{E}_{p(c|I, \theta_{old})}[\log p(c|I, \theta)]\end{aligned}$$

One can show that

$$0 \geq \mathbb{E}_{p(c|I, \theta_{old})}[\log p(c|I, \theta_{old})] \geq \mathbb{E}_{p(c|I, \theta_{old})}[\log p(c|I, \theta)], \quad \forall \theta$$

This means two things:

- $\log p(I|\theta) \geq Q(\theta|\theta_{old})$
- If we find a θ that increases $Q(\theta|\theta_{old})$ we also increase $\log p(I|\theta)$

Iterative algorithm with two step process

- **E-step:** Compute $p(c|I, \theta_{old})$ given θ_{old}
- **M-step:** Maximize $Q(\theta|\theta_{old})$, i.e. $\max_{\theta} \mathbb{E}_{p(c|I, \theta_{old})} [\log p(I, c|\theta)]$,
- Set $\theta_{old} = \theta^*$
- Start with a random θ_{old} , iterate E and M steps

For the Gaussian Mixture Model

E-step:

$$\begin{aligned} p(c(x)|I(x), \theta_{old}) &= \frac{p(I(x)|c(x), \theta_{old})p(c(x)|\theta_{old})}{p(I|\theta_{old})} \\ &= \frac{\mathcal{N}(I(x)|\mu_{c(x)}^{old}, \Sigma_{c(x)}^{old})\pi_{c(x)}^{old}}{\sum_{n=1}^N \mathcal{N}(I(x)|\mu_n^{old}, \Sigma_n^{old})\pi_n^{old}} \end{aligned}$$

For the Gaussian Mixture Model

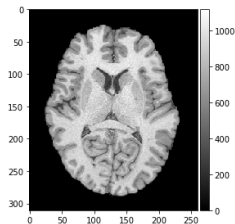
E-step:

$$\begin{aligned}
 p(c(x)|I(x), \theta_{old}) &= \frac{p(I(x)|c(x), \theta_{old})p(c(x)|\theta_{old})}{p(I|\theta_{old})} \\
 &= \frac{\mathcal{N}(I(x)|\mu_{c(x)}^{old}, \Sigma_{c(x)}^{old})\pi_{c(x)}^{old}}{\sum_{n=1}^N \mathcal{N}(I(x)|\mu_n^{old}, \Sigma_n^{old})\pi_n^{old}}
 \end{aligned}$$

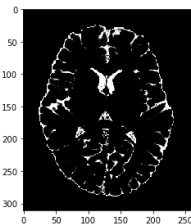
M-step:

$$\begin{aligned}
 \pi_n &= \frac{\sum_{x \in \Omega} p(c(x) = n|I(x), \theta_{old})}{|\Omega|} \\
 \mu_n &= \frac{\sum_{x \in \Omega} I(x)p(c(x) = n|I(x), \theta_{old})}{\sum_{x \in \Omega} p(c(x) = n|I(x), \theta_{old})} \\
 \Sigma_n &= \frac{\sum_{x \in \Omega} (I(x) - \mu_n)(I(x) - \mu_n)^T p(c(x) = n|I(x), \theta_{old})}{\sum_{x \in \Omega} p(c(x) = n|I(x), \theta_{old})}
 \end{aligned}$$

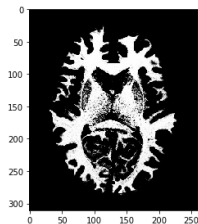
Examples - 3 components



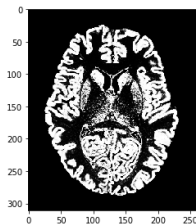
Image



$p(c = 1|I)$

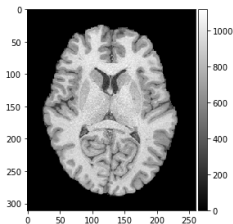


$p(c = 2|I)$

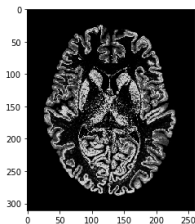
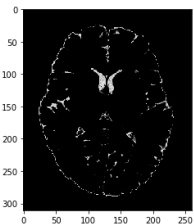
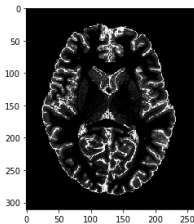
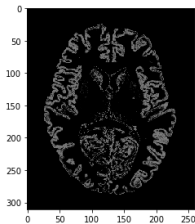
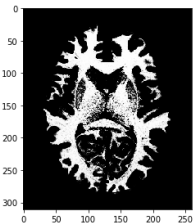


$p(c = 3|I)$

Examples - 5 components



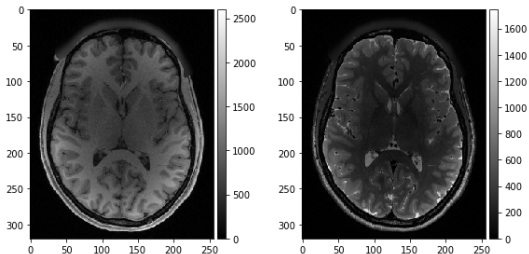
Image

 $p(c = 1 | I)$  $p(c = 2 | I)$  $p(c = 3 | I)$  $p(c = 4 | I)$  $p(c = 5 | I)$

Remarks

- Quite robust method
- Initialization can be important if done badly
- Number of components is important. You may need prior information regarding how many different structures you would like to extract from the images.
- Outputs already useful - gray matter density maps
- Used very regularly - in famous tools such as SPM, FSL and Freesurfer
- Extension to include better prior information, e.g., atlas, is possible.

Simple Segmentation Challenge



These two volumes are in the moodle platform - *Simple Segmentation Challenge*

They are T1-weighted and T2-weighted images of the same individual.

Goals

1. Perform bias removal in both images. Compare the bias fields. Are they different?
2. Perform EM segmentation on individual images and jointly. Compare segmentation results. Is using multiple modalities provide better defined clusters?

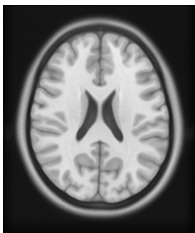
Section 2

Atlas-based segmentation

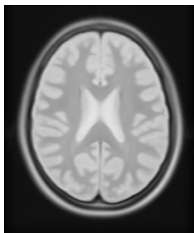
World atlas



Brain atlas



T1-weighted



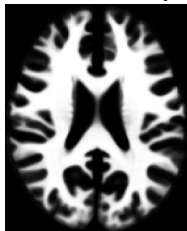
Proton Density



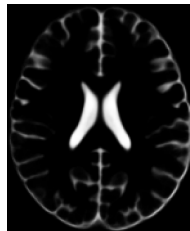
Brain



Gray matter



White matter

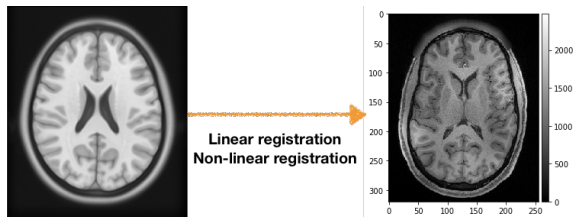


Cerebrospinal fluid

Atlas

- Adding prior information to the segmentation - no longer a simple clustering.
- Template at a reference frame
- For each location provides information about what is there
- Deterministic - world atlas
- Probabilistic, i.e. $p(c(x))$ - brain atlas
- Applicable to any anatomical structure
- Created by averaging many different images. The one shown before was made using 152 different images.

Aligning to a reference frame



- Align the atlas to the image
- Linear or non-linear registration
- The image is mapped on a reference frame
- At each point in the image now we have a rough idea what structure to find

Probabilistic formulation

- We start with the mixture model

$$\begin{aligned} p(I(x)|\theta) &= \sum_{n=1}^N p(I(x)|c(x) = n, \theta) p(c(x) = n|\theta) \\ &= \sum_{n=1}^N \mathcal{N}(I(x)|\mu_n, \Sigma_n) \pi_n \end{aligned}$$

where $\theta = \{\mu_n, \Sigma_n, \pi_n\}_{n=1}^N$. Parameters are determined in an unsupervised way.

Probabilistic formulation

- We start with the mixture model

$$\begin{aligned} p(I(x)|\theta) &= \sum_{n=1}^N p(I(x)|c(x) = n, \theta) p(c(x) = n|\theta) \\ &= \sum_{n=1}^N \mathcal{N}(I(x)|\mu_n, \Sigma_n) \pi_n \end{aligned}$$

where $\theta = \{\mu_n, \Sigma_n, \pi_n\}_{n=1}^N$. Parameters are determined in an unsupervised way.

- Add the atlas information

$$p(I(x)|\theta) = \sum_{n=1}^N \mathcal{N}(I(x)|\mu_n, \Sigma_n) p_{\text{atlas}}(c(x) = n)$$

where $\theta = \{\mu_n, \Sigma_n\}_{n=1}^N$. Atlas brings in prior information.

Posterior distribution takes into account the atlas

$$p(c(x) = n | I(x)) = \frac{\mathcal{N}(I(x) | \mu_n, \Sigma_n) p_{\text{atlas}}(c(x) = n)}{\sum_{n=1}^N \mathcal{N}(I(x) | \mu_n, \Sigma_n) p_{\text{atlas}}(c(x) = n)}$$

- Posterior takes into account the atlas and the image intensities

Posterior distribution takes into account the atlas

$$p(c(x) = n | I(x)) = \frac{\mathcal{N}(I(x) | \mu_n, \Sigma_n) p_{\text{atlas}}(c(x) = n)}{\sum_{n=1}^N \mathcal{N}(I(x) | \mu_n, \Sigma_n) p_{\text{atlas}}(c(x) = n)}$$

- Posterior takes into account the atlas and the image intensities
- Information propagation from the atlas to the image

Posterior distribution takes into account the atlas

$$p(c(x) = n | I(x)) = \frac{\mathcal{N}(I(x) | \mu_n, \Sigma_n) p_{\text{atlas}}(c(x) = n)}{\sum_{n=1}^N \mathcal{N}(I(x) | \mu_n, \Sigma_n) p_{\text{atlas}}(c(x) = n)}$$

- Posterior takes into account the atlas and the image intensities
- Information propagation from the atlas to the image
- No need to optimize for class probabilities $p(c(x) = n) = \pi_n$.

Posterior distribution takes into account the atlas

$$p(c(x) = n | I(x)) = \frac{\mathcal{N}(I(x) | \mu_n, \Sigma_n) p_{\text{atlas}}(c(x) = n)}{\sum_{n=1}^N \mathcal{N}(I(x) | \mu_n, \Sigma_n) p_{\text{atlas}}(c(x) = n)}$$

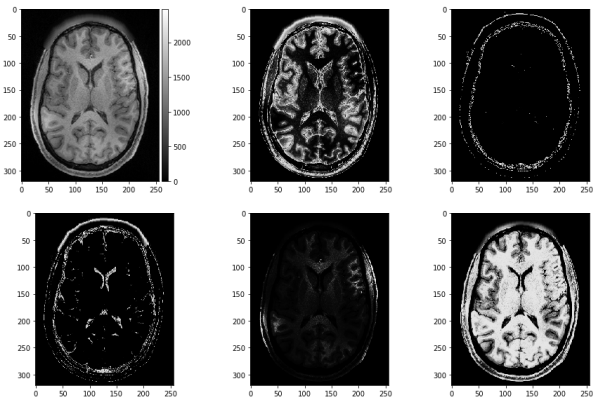
- Posterior takes into account the atlas and the image intensities
- Information propagation from the atlas to the image
- No need to optimize for class probabilities $p(c(x) = n) = \pi_n$.
- Do we still need to optimize for μ_n and Σ_n ?

Posterior distribution takes into account the atlas

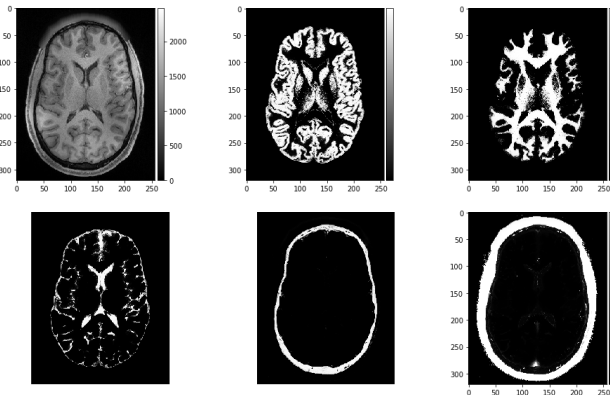
$$p(c(x) = n | I(x)) = \frac{\mathcal{N}(I(x) | \mu_n, \Sigma_n) p_{\text{atlas}}(c(x) = n)}{\sum_{n=1}^N \mathcal{N}(I(x) | \mu_n, \Sigma_n) p_{\text{atlas}}(c(x) = n)}$$

- Posterior takes into account the atlas and the image intensities
- Information propagation from the atlas to the image
- No need to optimize for class probabilities $p(c(x) = n) = \pi_n$.
- Do we still need to optimize for μ_n and Σ_n ?
- Yes for MRI!

How does it compare? - without Atlas



How does it compare? - with Atlas



Analysis

- Much better segmentation!
- No randomness in components. We know what we ask for.
- Used very commonly.
- Easily generalizable to other body parts with an appropriate atlas
- Can be generalized to other modalities. May fix all parameters for CT for instance.
- An outlier class can be added to detect outliers.

Analysis

- Much better segmentation!
- No randomness in components. We know what we ask for.
- Used very commonly.
- Easily generalizable to other body parts with an appropriate atlas
- Can be generalized to other modalities. May fix all parameters for CT for instance.
- An outlier class can be added to detect outliers.
- Need an atlas for each body part.
- Atlas itself is very important. Need an atlas for different age groups, gender, race...
- Accuracy of image registration is very important.

Two remedies

Two remedies to improve robustness to the atlas and registration

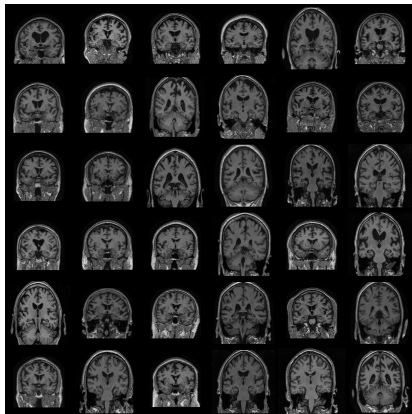
Multi-atlas segmentation

Instead of using one use many different atlases, propagate information from all of them and aggregate.

Patch matching

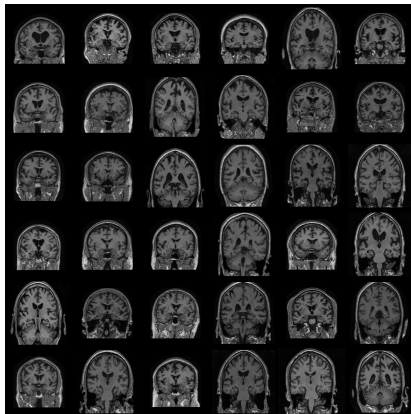
Use patch-matching to perform local searches in a sliding window manner.

Using many atlases rather than one



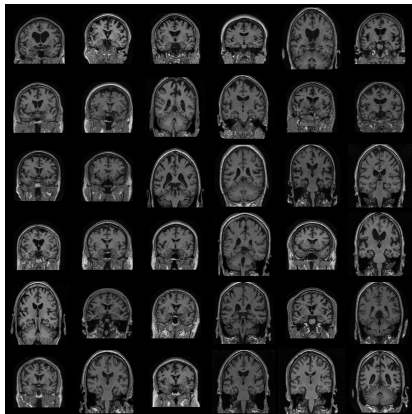
- Align all the atlas images to the test image

Using many atlases rather than one



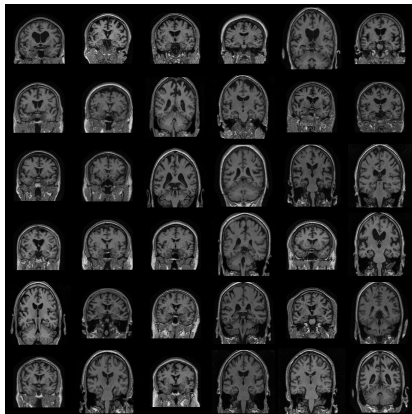
- Align all the atlas images to the test image
- Aggregate information coming from all the atlases

Using many atlases rather than one



- Align all the atlas images to the test image
- Aggregate information coming from all the atlases
- Remodeling $p_{\text{atlas}}(c(x) = n)$

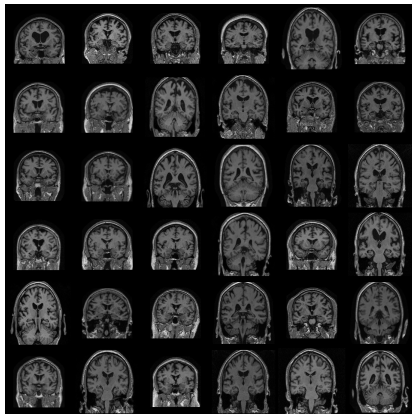
Using many atlases rather than one



- Align all the atlas images to the test image
- Aggregate information coming from all the atlases
- Remodeling $p_{\text{atlas}}(c(x) = n)$
- During aggregate weigh atlases based on the quality of registration

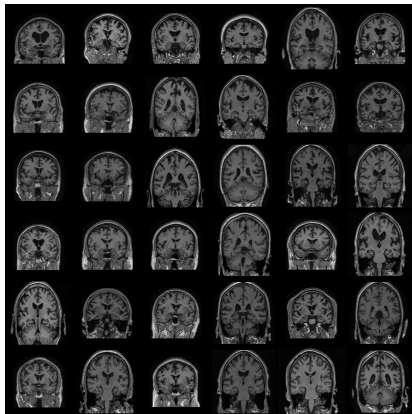
- $w_m \propto \|I(x) - A_m\|_2$
- $w_m \propto \|I(x) - T \circ A_m\|_2$
- $w_m \propto |J_{T_{A_m \rightarrow I}}|$

Using many atlases rather than one



- Align all the atlas images to the test image
- Aggregate information coming from all the atlases
- Remodeling $p_{\text{atlas}}(c(x) = n)$
- During aggregate weigh atlases based on the quality of registration
 - $w_m \propto \|I(x) - A_m\|_2$
 - $w_m \propto \|I(x) - T \circ A_m\|_2$
 - $w_m \propto |\mathbf{J}_{T_{A_m \rightarrow I}}|$
- Many atlases reduces importance of a single

Using many atlases rather than one



- Align all the atlas images to the test image
- Aggregate information coming from all the atlases
- Remodeling $p_{\text{atlas}}(c(x) = n)$
- During aggregate weigh atlases based on the quality of registration
 - $w_m \propto \|I(x) - A_m\|_2$
 - $w_m \propto \|I(x) - T \circ A_m\|_2$
 - $w_m \propto |J_{T_{A_m \rightarrow I}}|$
- Many atlases reduces importance of a single
- Less likely to have bad registration with weighing scheme

Analysis

- Solves some of the problems of atlas-based segmentation
- Highly accurate results
- State-of-the-art for a very long time

Analysis

- Solves some of the problems of atlas-based segmentation
- Highly accurate results
- State-of-the-art for a very long time
- Leads to elegant probabilistic models
- [Multi-atlas segmentation of biomedical images: A survey, J.Iglesias, M.Sabuncu, Medical Image Analysis, 2015]

Analysis

- Solves some of the problems of atlas-based segmentation
- Highly accurate results
- State-of-the-art for a very long time
- Leads to elegant probabilistic models
- [Multi-atlas segmentation of biomedical images: A survey, J.Iglesias, M.Sabuncu, Medical Image Analysis, 2015]
- Often requires non-linear registration
- Computationally **VERY** expensive due to many registrations

Section 3

Spatial constraints in segmentation

No spatial information so far

- All the pixels are independent
- Atlases include some spatial information but the final segmentation still assumed independence.
- In real images this is a highly unreasonable assumption
- Neighboring pixels are more likely to belong to the same object

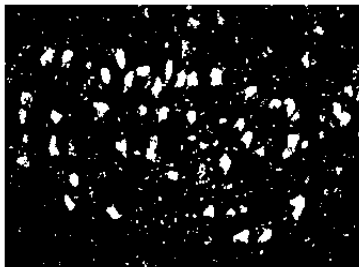
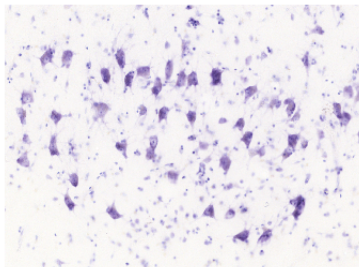
No spatial information so far

- All the pixels are independent
- Atlases include some spatial information but the final segmentation still assumed independence.
- In real images this is a highly unreasonable assumption
- Neighboring pixels are more likely to belong to the same object

Two main approaches to integrate spatial information

- Pre/Post-processing
 - Smoothing the images before hand
 - Morphological operations
- Random field priors
 - Markov random fields
 - Conditional random fields

Operations on binary images



- Isolated islands
- Broken structures
- What causes them: Noise, low contrast boundaries, intensity overlap between structures,...
- Operations on the segmentation image
- Remove islands
- Fill in gaps

Basic operations in mathematical morphology

- Shift-invariant
- Non-linear
- Based on neighboring pixels defined through structural elements
- Applied in a sliding window approach, e.g. structural element slides across the image
- View binary images as sets of pixels
- Defined in binary but extended to gray-level images
- Two main operations

Dilation

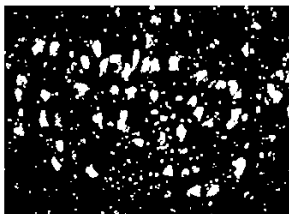
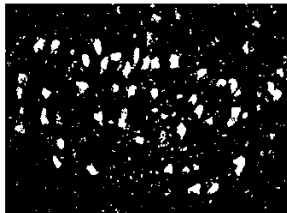
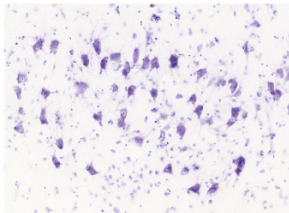
$$C = A \oplus B \triangleq \{x | B_x \cap A \neq \emptyset\}$$

A is the image and B is a structural element, e.g. $\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$

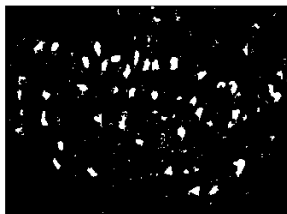
Erosion

$$C = A \ominus B \triangleq \{x | B_x \subseteq A\}$$

Dilation and Erosion - $B = \mathbf{1}_{7 \times 7}$

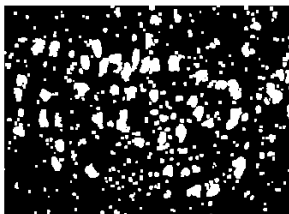
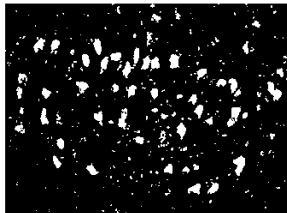
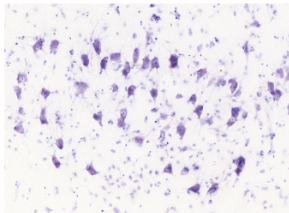


Dilated



Eroded

Dilation and Erosion - $B = \mathbf{1}_{13 \times 13}$



Dilated



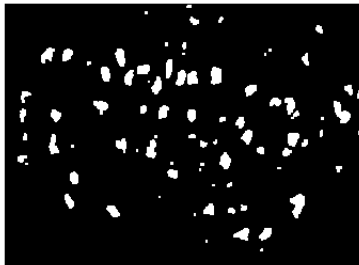
Eroded

Derived operations

- Combining the two to get useful operations

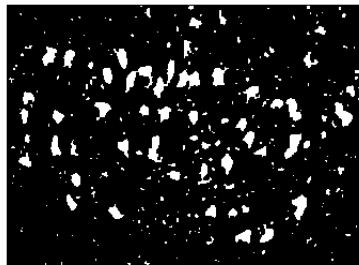
Opening

$$(A \ominus B) \oplus B$$

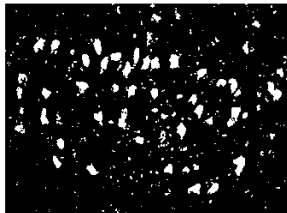
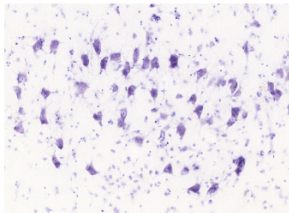


Closing

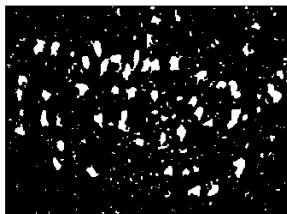
$$(A \oplus B) \ominus B$$



Opening and Closing are useful - $B = \mathbf{1}_{13 \times 13}$



Opening



Closing

No spatial information so far

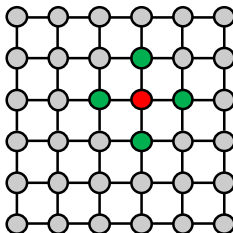
- All the pixels are independent
- Atlases include some spatial information but the final segmentation still assumed independence.
- In real images this is a highly unreasonable assumption
- Neighboring pixels are more likely to belong to the same object

Two main approaches to integrate spatial information

- Pre/Post-processing
 - Smoothing the images before hand
 - Morphological operations
- Random field priors
 - Markov random fields
 - Conditional random fields

Neighborhood consistency

- Formulating neighborhood consistency in a probabilistic model / energy model



- Main idea is to punish inconsistency in labeling of neighboring voxels
- We will use the Markovian property to define the punishment
- Let's start with the probabilistic formulation

Markov Random Fields: principle

- So far our probabilistic models were mostly a mixture model. Let's focus on the joint distribution of labels and intensities

$$p(I(x)) = \sum_{n=1}^N p(I(x)|c(x) = n)p(c(x) = n)$$

Markov Random Fields: principle

- So far our probabilistic models were mostly a mixture model. Let's focus on the joint distribution of labels and intensities

$$\begin{aligned}p(I(x)) &= \sum_{n=1}^N p(I(x)|c(x) = n)p(c(x) = n) \\p(I(x), c(x)) &= p(I(x)|c(x))p(c(x))\end{aligned}$$

Markov Random Fields: principle

- So far our probabilistic models were mostly a mixture model. Let's focus on the joint distribution of labels and intensities

$$p(I(x)) = \sum_{n=1}^N p(I(x)|c(x) = n)p(c(x) = n)$$

$$p(I(x), c(x)) = p(I(x)|c(x))p(c(x))$$

$$p(I, c) = \prod_{x \in \Omega} p(I(x)|c(x))p(c(x))$$

where all pixels are independent.

Markov Random Fields: principle

- So far our probabilistic models were mostly a mixture model. Let's focus on the joint distribution of labels and intensities

$$\begin{aligned}p(I(x)) &= \sum_{n=1}^N p(I(x)|c(x) = n)p(c(x) = n) \\p(I(x), c(x)) &= p(I(x)|c(x))p(c(x)) \\p(I, c) &= \prod_{x \in \Omega} p(I(x)|c(x))p(c(x))\end{aligned}$$

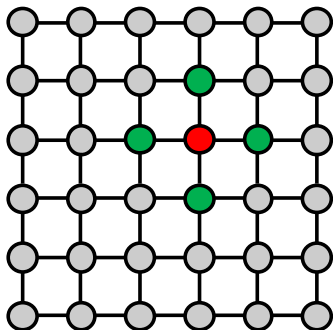
where all pixels are independent.

- In order to model connection between pixels we change the prior

$$p(I, c) = p(c) \prod_{x \in \Omega} p(I(x)|c(x)).$$

Joint distribution changes.

Markovian property



- MRF sets up the prior distribution based on the Markovian property
- Specific form depends on the neighborhood structure

$G(x)$ = neighbors of x

$$p(c(x)|c(/x)) = p(c(x)|c(G(x)))$$

$p(c(x))$ only depends on its immediate neighbors if all others are given.

Energy formulation

$$p(c(x)|c(/x)) = p(c(x)|c(G(x)))$$

- A common way to define it is through defining an energy:

$$E(c) = \sum_{x \in \Omega} \sum_{y \in G(x)} d(c(x), c(y))$$

$d(c(x), c(y))$ is a distance between the labels.

- Given the energy we can define a probability distribution

$$p(c) = \frac{1}{Z} \exp \{-E(c)\}$$

Z is the normalization constant. Lower distance means higher probability. It enforces consistency.

Gibbs distribution

$$E(c) = \sum_{x \in \Omega} \sum_{y \in G(x)} d(c(x), c(y)) \longleftrightarrow p(c) = \frac{1}{Z} \exp \{-E(c)\}$$

Is this even allowed?

Gibbs distribution

$$E(c) = \sum_{x \in \Omega} \sum_{y \in G(x)} d(c(x), c(y)) \longleftrightarrow p(c) = \frac{1}{Z} \exp \{-E(c)\}$$

Is this even allowed?

Hammersley-Clifford Theorem

Any probability distribution that satisfies a Markovian property is a Gibbs distribution for an appropriate locally-defined energy and vice-versa.

Segmentation via posterior maximization

Given the energy function $E(c)$

$$p(c) = \frac{1}{Z} \exp \{-E(c)\}$$

Posterior distribution becomes

$$p(c|I) = \frac{p(I|c)p(c)}{p(I)}$$

Note that the posterior is no longer point-wise. It is a joint distribution.

Segmentation via posterior maximization

Given the energy function $E(c)$

$$p(c) = \frac{1}{Z} \exp \{-E(c)\}$$

Posterior distribution becomes

$$p(c|I) = \frac{p(I|c)p(c)}{p(I)}$$

Note that the posterior is no longer point-wise. It is a joint distribution.
Segmentation via posterior maximization:

$$\begin{aligned} \arg_c \max p(c|I) &= \arg_c \max p(I|c)p(c) \\ &= \arg_c \max \exp\{-E(c)\} \prod_{x \in \Omega} p(I(x)|c(x)) \end{aligned}$$

assuming given the label of a voxel, its intensity is independent from the intensities of the other voxels.

Posterior maximization - energy minimization

$$\arg_c \max p(c|I) = \arg_c \max \exp\{-E(c)\} \prod_{x \in \Omega} p(I(x)|c(x))$$

Posterior maximization - energy minimization

$$\arg_c \max p(c|I) = \arg_c \max \exp\{-E(c)\} \prod_{x \in \Omega} p(I(x)|c(x))$$

If the data model is also exponential of another energy

$$p(I(x)|c(x)) \propto \exp\{-g(I(x)|\theta_{c(x)})\}$$

Posterior maximization - energy minimization

$$\arg_c \max p(c|I) = \arg_c \max \exp\{-E(c)\} \prod_{x \in \Omega} p(I(x)|c(x))$$

If the data model is also exponential of another energy

$$p(I(x)|c(x)) \propto \exp\{-g(I(x)|\theta_{c(x)})\}$$

Then we end up with an energy minimization formulation

$$\begin{aligned} \arg_c \max p(c|I) &= \arg_c \max \ln p(c|I) \\ &= \arg_c \max \left\{ -\sum_{x \in \Omega} g(I(x)|\theta_{c(x)}) - \sum_{x \in \Omega} \sum_{y \in G(x)} d(c(x), c(y)) \right\} \\ &= \arg_c \min \left\{ \sum_{x \in \Omega} g(I(x)|\theta_{c(x)}) + \sum_{x \in \Omega} \sum_{y \in G(x)} d(c(x), c(y)) \right\} \end{aligned}$$

Different terms

$$\arg_c \min \underbrace{\sum_{x \in \Omega} g(I(x) | \theta_{c(x)})}_{\text{Unary term}} + \underbrace{\sum_{x \in \Omega} \sum_{y \in G(x)} d(c(x), c(y))}_{\text{Pairwise term}}$$

- Unary term is also the data term in this model. Sometimes called data fidelity term.
- However it could also include any unary prior information on the labels.
- Pairwise term is the consistency term that imposes neighboring pixels to have similar labels.
- When $c(x)$ is a continuous variable, as in regression problems, the optimization can be tackled with usual techniques.
- When $c(x)$ is a categorical variable, e.g., segmentation, then the optimization can be challenging.

Simple example

Observation model

$$p(I(x)|c(x)) = \mathcal{N}(I(x)|\mu_{c(x)}, \Sigma_{c(x)})$$

where $g(I(x)|\theta_{c(x)}) = (I(x) - \mu_{c(x)})^T \Sigma_{c(x)}^{-1} (I(x) - \mu_{c(x)})$. Energy model for the prior distribution - **Ising / Potts Model**

$$d(c(x), c(y)) = \lambda \delta(c(x) \neq c(y)) = \begin{cases} 0, & c(x) = c(y) \\ 1, & c(x) \neq c(y) \end{cases}$$

Segmentation is the solution of

$$\arg_c \min \sum_{x \in \Omega} (I(x) - \mu_{c(x)})^T \Sigma_{c(x)}^{-1} (I(x) - \mu_{c(x)}) + \lambda \sum_{x \in \Omega} \sum_{y \in G(x)} \delta(c(x) \neq c(y))$$

where λ becomes a trade-off parameter between data fidelity and neighborhood consistency.

Optimization

$$p(c|I) = \frac{p(I|c)p(c)}{p(I)}$$

$$\arg_c \max p(I|c)p(c)$$

$$\arg_c \min \sum_{x \in \Omega} (I(x) - \mu_{c(x)})^T \Sigma_{c(x)}^{-1} (I(x) - \mu_{c(x)}) + \lambda \sum_{x \in \Omega} \sum_{y \in G(x)} \delta(c(x) \neq c(y))$$

- Very difficult to compute the posterior distribution
- Difficult to solve the optimization exactly in 2D and higher
- NP-hard [Boykov, Veksler and Zabih 2001 TPAMI]
- Approximate energy minimization and posterior sampling
 - Gibbs sampling [Geman and Geman 1984]
 - Iterated conditional models (ICM) [Ferrari et al. 1995]
 - Graph Cuts [Boykov, Veksler and Zabih 2001]
 - ...

Stochastic relaxation / Gibbs sampling: principle

- Iterative method to approximate solution
- Monte-Carlo sampling method
- Sample from the posterior distribution is hard!

$$p(c|I) = \frac{p(I|c)p(c)}{p(I)} = \frac{p(I|c)p(c)}{\sum_c p(I|c)p(c)}$$

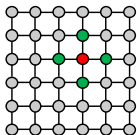
Stochastic relaxation / Gibbs sampling: principle

- Iterative method to approximate solution
- Monte-Carlo sampling method
- Sample from the posterior distribution is hard!

$$p(c|I) = \frac{p(I|c)p(c)}{p(I)} = \frac{p(I|c)p(c)}{\sum_c p(I|c)p(c)}$$

- Much easier to sample from this

$$p(c(x)|c(\setminus x), I) = \frac{p(I(x)|c(x)) p(c(x)|c(G(x)))}{\sum_{c(x)} p(I(x)|c(x)) p(c(x)|c(G(x)))}$$



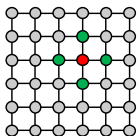
Stochastic relaxation / Gibbs sampling: principle

- Iterative method to approximate solution
- Monte-Carlo sampling method
- Sample from the posterior distribution is hard!

$$p(c|I) = \frac{p(I|c)p(c)}{p(I)} = \frac{p(I|c)p(c)}{\sum_c p(I|c)p(c)}$$

- Much easier to sample from this

$$p(c(x)|c(/x), I) = \frac{p(I(x)|c(x)) p(c(x)|c(G(x)))}{\sum_{c(x)} p(I(x)|c(x)) p(c(x)|c(G(x)))}$$



- Start from a random c_0 assignment, such as $\arg_c \max \prod_{x \in \Omega} p(I(x)|c(x))$

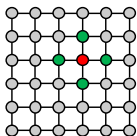
Stochastic relaxation / Gibbs sampling: principle

- Iterative method to approximate solution
- Monte-Carlo sampling method
- Sample from the posterior distribution is hard!

$$p(c|I) = \frac{p(I|c)p(c)}{p(I)} = \frac{p(I|c)p(c)}{\sum_c p(I|c)p(c)}$$

- Much easier to sample from this

$$p(c(x)|c(\setminus x), I) = \frac{p(I(x)|c(x)) p(c(x)|c(G(x)))}{\sum_{c(x)} p(I(x)|c(x)) p(c(x)|c(G(x)))}$$



- Start from a random c_0 assignment, such as $\arg_c \max \prod_{x \in \Omega} p(I(x)|c(x))$
- Sample from the posterior of each voxel given the others

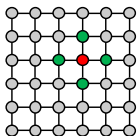
Stochastic relaxation / Gibbs sampling: principle

- Iterative method to approximate solution
- Monte-Carlo sampling method
- Sample from the posterior distribution is hard!

$$p(c|I) = \frac{p(I|c)p(c)}{p(I)} = \frac{p(I|c)p(c)}{\sum_c p(I|c)p(c)}$$

- Much easier to sample from this

$$p(c(x)|c(\setminus x), I) = \frac{p(I(x)|c(x)) p(c(x)|c(G(x)))}{\sum_{c(x)} p(I(x)|c(x)) p(c(x)|c(G(x)))}$$



- Start from a random c_0 assignment, such as $\arg_c \max \prod_{x \in \Omega} p(I(x)|c(x))$
- Sample from the posterior of each voxel given the others
- Iterate over all voxels and some number of times

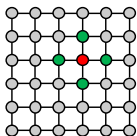
Stochastic relaxation / Gibbs sampling: principle

- Iterative method to approximate solution
- Monte-Carlo sampling method
- Sample from the posterior distribution is hard!

$$p(c|I) = \frac{p(I|c)p(c)}{p(I)} = \frac{p(I|c)p(c)}{\sum_c p(I|c)p(c)}$$

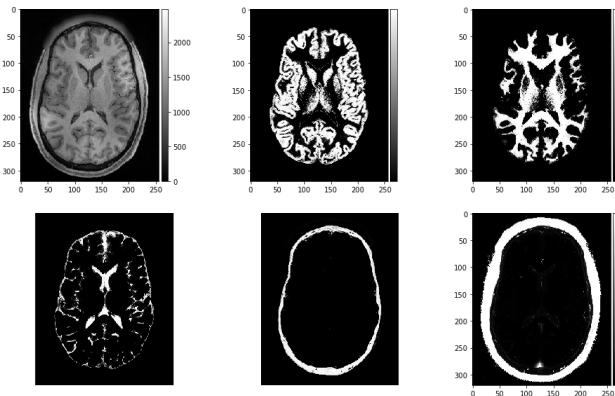
- Much easier to sample from this

$$p(c(x)|c(/x), I) = \frac{p(I(x)|c(x)) p(c(x)|c(G(x)))}{\sum_{c(x)} p(I(x)|c(x)) p(c(x)|c(G(x)))}$$

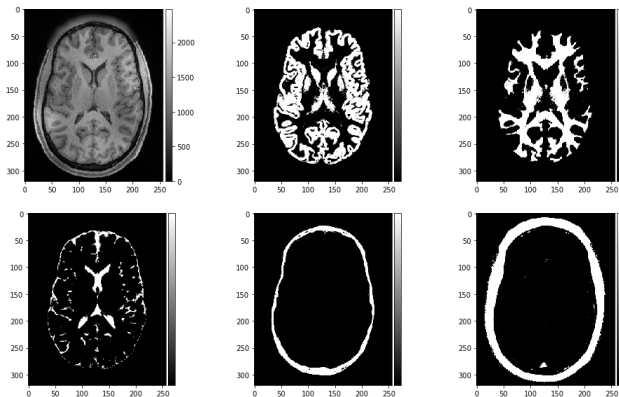


- Start from a random c_0 assignment, such as $\arg_c \max \prod_{x \in \Omega} p(I(x)|c(x))$
- Sample from the posterior of each voxel given the others
- Iterate over all voxels and some number of times
- Convergence properties [Geman and Geman 1984]

Example results - without MRF

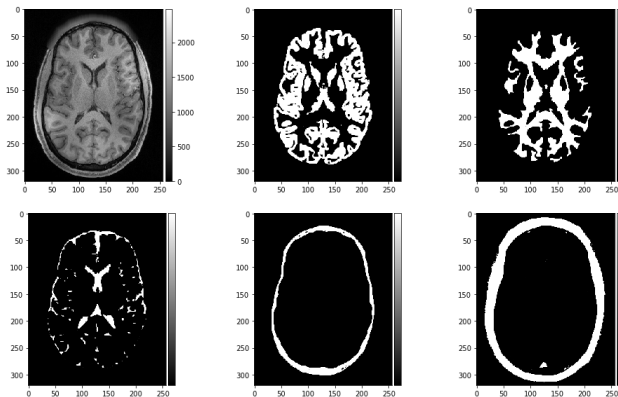


Example results - with MRF



- └ Spatial constraints in segmentation
- └ Random field priors

Example results - with MRF higher λ



Analysis of Gibbs Sampling

- Very simple implementation and effective
- Solution depends on parameters
- Solution depends on initial conditions
- May not converge to a pleasing result
- Convergence may be slow
- Stochastic in nature - every time you run you get something else
- Other methods exist and may provide better results
- Complicated optimization problem

Conditional random fields

- The energy formulation of MRF's can also be used in a discriminative way
- Instead of a probabilistic generative model we can consider a model that directly predicts $p(c(x)|I)$ without the Bayesian treatment

$$\arg_c \min \underbrace{\sum_{x \in \Omega} g(c(x)|I)}_{\text{Unary term}} + \underbrace{\sum_{x \in \Omega} \sum_{y \in G(x)} d(c(x), c(y))}_{\text{Pairwise term}}$$

- This can be used on its own for segmentation by explicitly modeling g
- It can also be used with ML algorithm outputs
- Unary term can be the predictions of a random forest
- Cleaning up the ML segmentation results
- Used regularly